

paper_analysis

September 2, 2026

1 Efficient Predator–Prey Paper Analysis

One tensor workflow for every available Couzin/biological expert/imitation case; unavailable cases are skipped cleanly.

The notebook follows the analysis pipeline in execution order: configuration, case preparation, shared metric computation, metric-specific visualizations, and finally the modular-network comparison. Expensive trajectories and derived results are calculated once and then reused.

```
[1]: # Standard-library, numerical, plotting, and notebook-display imports.
from pathlib import Path
import html, math, sys
import numpy as np
import torch
import matplotlib.pyplot as plt
from tqdm.auto import tqdm
from IPython.display import HTML, display

# Resolve project paths whether the notebook starts in the project root or this
↳ folder.
ANALYSIS_DIR = Path.cwd().resolve()
if ANALYSIS_DIR.name != 'Paper Analysis': ANALYSIS_DIR = ANALYSIS_DIR / 'Paper_
↳ Analysis'
PROJECT_ROOT, NICOLE_ROOT = ANALYSIS_DIR.parent, ANALYSIS_DIR.parent / 'Nicole'
↳ / 'Predator-Prey-Thesis'
OUTPUT_DIR = ANALYSIS_DIR / 'outputs'

# Import the local analysis helpers without changing either source repository.
if str(ANALYSIS_DIR) not in sys.path: sys.path.insert(0, str(ANALYSIS_DIR))
import paper_analysis as pa
```

```
c:\Users\janni\anaconda3\envs\test_gail\lib\site-packages\tqdm\auto.py:21:
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
```

1.1 Configuration

This section contains the small set of global analysis choices. Thresholds and risk bins are shared across every source, policy, and group size so comparisons use identical definitions.

```
[2]: # Tensor execution settings.
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
DTYPE = torch.float32

# Validated shared approach and response definitions.
APPROACH_DISTANCE_THRESHOLD, APPROACH_CLOSING_LAG = 0.15, 3
CLOSING_SPEED_LAG = 3
RESPONSE_THRESHOLD, RESPONSE_WINDOW_STEPS = 0.10, 20
RESPONSE_THRESHOLD_SENSITIVITY = (0.075, 0.10, 0.125)
RESPONSE_LAG, TIME_CHUNK_SIZE = 1, 4096

# Source-specific central q01-q99 Expert-Imitation overlap from data_validation.
↳ ipynb.
RISK_RANGES = {'couzin': (0.0144023038, 0.2365715653), 'biological': (0.
↳ 0307952035, 0.3913621008)}
N_DISTANCE_BINS, MIN_SAMPLES_PER_BIN, MIN_CLIPS_PER_BIN = 10, 30, 5
N_BOOTSTRAP, CI_LEVEL = 1000, 0.95
RISK_BIN_EDGES = {source: torch.
↳ linspace(low, high, N_DISTANCE_BINS+1, device=DEVICE, dtype=DTYPE) for
↳ source, (low, high) in RISK_RANGES.items()}

# Source-specific arena diagonals used for distance normalization.
D_SOURCE = {'couzin': math.hypot(50, 50), 'biological': math.hypot(2160, 2160)}

# Rollout generation and optional result persistence.
N_ROLLOUTS, ROLLOUT_STEPS, SEED, SAVE_RESULTS = 20, 1000, 2027, False
ANALYSIS_SIGNATURE = {'approach_threshold':
↳ APPROACH_DISTANCE_THRESHOLD, 'approach_lag':
↳ APPROACH_CLOSING_LAG, 'closing_lag': CLOSING_SPEED_LAG, 'response_threshold':
↳ RESPONSE_THRESHOLD, 'response_threshold_sensitivity':
↳ RESPONSE_THRESHOLD_SENSITIVITY, 'response_window':
↳ RESPONSE_WINDOW_STEPS, 'n_bins': N_DISTANCE_BINS, 'min_samples':
↳ MIN_SAMPLES_PER_BIN, 'min_clips': MIN_CLIPS_PER_BIN, 'risk_ranges': RISK_RANGES}
print({'device': str(DEVICE), 'D_source': D_SOURCE, 'risk_ranges': RISK_RANGES})

{'device': 'cuda', 'D_source': {'couzin': 70.71067811865476, 'biological':
3054.7012947258854}, 'risk_ranges': {'couzin': (0.0144023038, 0.2365715653),
'biological': (0.0307952035, 0.3913621008)}}
```

1.2 Load or generate all cases once

D_source is the declared arena diagonal, never an observed trajectory maximum. Nearest prey is recomputed every frame and is only a target proxy.

The workspace currently has no continuous biological 16-prey expert trajectory. That intended case is reported under `skipped_cases` without a warning; init pools and overlapping training windows are deliberately not treated as expert sequences.

```
[3]: # Checkpoint and environment settings for the Couzin-trained policy pair.
COUZIN_POLICY = {
    'root': NICOLE_ROOT, 'code_subdir': '.',
    'prey_checkpoint': 'Data/2. Training/CouzinPredPrey - GAIL/
    ↪stage1_seed42_32and16/prey_policy_stage1.pth',
    'pred_checkpoint': 'Data/2. Training/CouzinPredPrey - GAIL/
    ↪stage1_seed42_32and16/pred_policy_stage1.pth',
    'prey_features': 6, 'pred_features': 5, 'deterministic': False,
    'environment': {'area_width':50,'area_height':50,'prey_speed':5,'pred_speed':
    ↪5,'step_size':0.5,'max_turn':0.25,'max_speed_norm':5.0}}

# Checkpoint and environment settings for the biologically trained policy pair.
BIO_POLICY = {
    'root': NICOLE_ROOT, 'code_subdir': '.',
    'prey_checkpoint': 'Data/2. Training/VideoPredPrey - GAIL/
    ↪stage1_seed42_32and16/prey_policy_stage1.pth',
    'pred_checkpoint': 'Data/2. Training/VideoPredPrey - GAIL/
    ↪stage1_seed42_32and16/pred_policy_stage1.pth',
    'prey_features': 6, 'pred_features': 5, 'deterministic': False,
    'environment': {'area_width':2160,'area_height':2160,'prey_speed':
    ↪10,'pred_speed':10,'step_size':1.0,'max_turn':0.314,'max_speed_norm':25.0}}

# Existing expert-trajectory and initialization-pool locations.
BIO_EXPERT_ROOTS = {n: NICOLE_ROOT / f'Data/1. Data Processing/Processed/video/
    ↪expert_tensors/{n}_prey_interactions/2. filtered_frames' for n in (16,32)}
INIT_POOLS = {n: NICOLE_ROOT / f'Data/1. Data Processing/Processed/init_pool/
    ↪init_pool_{n}prey.pt' for n in (16,32)}

# Declare all intended source × policy type × group-size combinations in one_
    ↪place.
case_specs = {
    **{f'couzin_expert_{n}': {'source':'couzin','kind':'couzin_expert','n_prey':n}_
    ↪for n in (16,32)},
    **{f'couzin_imitation_{n}': {'source':'couzin','kind':'imitation','n_prey':
    ↪n,'policy':COUZIN_POLICY} for n in (16,32)},
    **{f'bio_expert_{n}': {'source':'biological','kind':'bio_expert','n_prey':n}_
    ↪for n in (16,32)},
    **{f'bio_imitation_{n}': {'source':'biological','kind':'imitation','n_prey':
    ↪n,'policy':BIO_POLICY} for n in (16,32)}}

# Load expert data or generate imitation rollouts exactly once per case.
cases, skipped_cases, case_diagnostics = {}, {}, {}
```

```

for offset, (name, spec) in enumerate(tqdm(case_specs.items(), desc='Loading/
↳generating cases')):
    n = spec['n_pre']
    if spec['kind'] == 'bio_expert' and not BIO_EXPERT_ROOTS[n].is_dir():
        skipped_cases[name] = 'No continuous expert trajectory is present (init pools/
↳windows are not valid substitutes).'
        continue
    try:
        if spec['kind'] == 'couzin_expert':
            table = pa.
↳generate_couzin_expert_rollouts(NICOLE_ROOT,n_pre=n,n_rollouts=N_ROLLOUTS,rollout_steps=RO
        elif spec['kind'] == 'bio_expert':
            table = pa.
↳load_expert_trajectories(BIO_EXPERT_ROOTS[n],n_pre=n,condition='all',image_y_down=True,coo
↳'predator','2':'prey'},require_complete_frames=True)
        else:
            config = spec['policy']; init_pool = None if spec['source'] == 'couzin' else
↳INIT_POOLS[n]
            table = pa.
↳generate_policy_rollouts(config,config['root'],n_pre=n,n_rollouts=N_ROLLOUTS,rollout_steps
    except (FileNotFoundError, pa.ExpertDataUnavailable) as exc:
        skipped_cases[name] = str(exc)
        continue

    # Validate every loaded case before it enters the shared analysis workflow.
    diag = pa.trajectory_diagnostics(table)
    if diag['prey_per_frame_min'] != n or diag['prey_per_frame_max'] != n:
        raise ValueError(f'{name}: expected exactly {n} prey in every frame, got
↳{diag}')
    if not np.all(np.isfinite(table['x'])) or not np.all(np.isfinite(table['y']))
↳or not np.all(np.isfinite(table['heading'])):
        raise ValueError(f'{name}: non-finite position or heading detected')
    cases[name], case_diagnostics[name] = table, diag

    # Show one concise availability overview and the diagnostics of usable cases.
    display({'available_cases': sorted(cases), 'skipped_cases': skipped_cases})
    display(case_diagnostics)

```

Loading/generating cases: 100% | 8/8 [11:48<00:00, 88.61s/it]

```

{'available_cases': ['bio_expert_32',
    'bio_imitation_16',
    'bio_imitation_32',
    'couzin_expert_16',
    'couzin_expert_32',
    'couzin_imitation_16',
    'couzin_imitation_32'],

```

```

'skipped_cases': {'bio_expert_16': 'No continuous expert trajectory is present_
↳(init pools/windows are not valid substitutes).'}}

{'couzin_expert_16': {'rows': 340000,
  'clips_or_rollouts': 20,
  'frames': 20000,
  'prey_per_frame_min': 16,
  'prey_per_frame_max': 16,
  'x_range': (-2.4999999890285083, 52.49991394902054),
  'y_range': (-2.4999871195112235, 52.49996006229406),
  'heading_range': (-3.141582229434665, 3.141584526483916),
  'conditions': {np.str_('approach'): 340000},
  'fps_values': [],
  'timestep_status': 'PHYSICAL TIME UNKNOWN'},
'couzin_expert_32': {'rows': 660000,
  'clips_or_rollouts': 20,
  'frames': 20000,
  'prey_per_frame_min': 32,
  'prey_per_frame_max': 32,
  'x_range': (-2.499974720310492, 52.49998754799828),
  'y_range': (-2.4999779197080216, 52.499825518459616),
  'heading_range': (-3.1415718584578896, 3.1415784160929654),
  'conditions': {np.str_('approach'): 660000},
  'fps_values': [],
  'timestep_status': 'PHYSICAL TIME UNKNOWN'},
'couzin_imitation_16': {'rows': 340000,
  'clips_or_rollouts': 20,
  'frames': 20000,
  'prey_per_frame_min': 16,
  'prey_per_frame_max': 16,
  'x_range': (-2.4981211870908737, 52.49515771865845),
  'y_range': (-2.4963371455669403, 52.49549150466919),
  'heading_range': (-3.141589403152466, 3.141576051712036),
  'conditions': {np.str_('unknown'): 340000},
  'fps_values': [],
  'timestep_status': 'PHYSICAL TIME UNKNOWN'},
'couzin_imitation_32': {'rows': 660000,
  'clips_or_rollouts': 20,
  'frames': 20000,
  'prey_per_frame_min': 32,
  'prey_per_frame_max': 32,
  'x_range': (-2.499382197856903, 52.49626636505127),
  'y_range': (-2.4931658059358597, 52.49384045600891),
  'heading_range': (-3.141589641571045, 3.1415750980377197),
  'conditions': {np.str_('unknown'): 660000},
  'fps_values': [],
  'timestep_status': 'PHYSICAL TIME UNKNOWN'},
'bio_expert_32': {'rows': 392007,

```

```

'clips_or_rollouts': 35,
'frames': 11879,
'prey_per_frame_min': 32,
'prey_per_frame_max': 32,
'x_range': (12.856284846417006, 2160.2542277168986),
'y_range': (0.0, 2146.369623144862),
'heading_range': (-3.14158531800636, 3.141589390613703),
'conditions': {np.str_('interaction'): 392007},
'fps_values': [],
'timestep_status': 'PHYSICAL TIME UNKNOWN'},
'bio_imitation_16': {'rows': 340000,
'clips_or_rollouts': 20,
'frames': 20000,
'prey_per_frame_min': 16,
'prey_per_frame_max': 16,
'x_range': (-254.28035080432892, 2169.6441078186035),
'y_range': (-9.95963416993618, 2436.1983489990234),
'heading_range': (-3.141465425491333, 3.1415598392486572),
'conditions': {np.str_('unknown'): 340000},
'fps_values': [],
'timestep_status': 'PHYSICAL TIME UNKNOWN'},
'bio_imitation_32': {'rows': 660000,
'clips_or_rollouts': 20,
'frames': 20000,
'prey_per_frame_min': 32,
'prey_per_frame_max': 32,
'x_range': (-492.7710950374603, 2583.35506439209),
'y_range': (-43.05211007595062, 2422.5940132141113),
'heading_range': (-3.141592502593994, 3.141591787338257),
'conditions': {np.str_('unknown'): 660000},
'fps_values': [],
'timestep_status': 'PHYSICAL TIME UNKNOWN'}}

```

```

[4]: # Run the complete metric suite once for every available case.
case_results = {}
with torch.inference_mode():
    for name, table in tqdm(cases.items(), desc='Analyzing cases'):
        source = case_specs[name]['source']
        case_results[name] = pa.
        ↪analyze_case(table,d_source=D_SOURCE[source],risk_bin_edges=RISK_BIN_EDGES[source],expected

# Calculate explicit 16-to-32 changes wherever a complete quartet exists.
group_size_results = pa.compute_all_group_size_comparisons(case_results)

# Build a metric-first view for later figure construction and serialization.
results = {}

```

```

for metric in
    ('closing_speed', 'pursuit_alignment', 'escape_alignment', 'risk_conditioned_nnd', 'risk_condit
    )
    results[metric] = {name:r['curves'][metric] for name,r in case_results.items()}
for metric in next(iter(case_results.values()))['aggregate'] if case_results
    else ():
    results.setdefault(metric, {}).update({name:r['aggregate'][metric] for name,r
    in case_results.items()})
results['events'] = {name:r['events'] for name,r in case_results.items()}
results['response_availability'] = {name:r['response_availability'] for name,r
    in case_results.items()}
results['predator_distance_raw'] = {name:r['predator_distance_raw'] for name,r
    in case_results.items()}
results['predator_distance_normalized'] = {name:
    r['predator_distance_normalized'] for name,r in case_results.items()}
results['diagnostics'] = {name:r['diagnostics'] for name,r in case_results.
    items()}
results['response_threshold_sensitivity'] = {name:
    r['response_threshold_sensitivity'] for name,r in case_results.items()}
results['group_size'] = group_size_results
results['analysis_signature'] = ANALYSIS_SIGNATURE

# Concise segmentation and event-consistency diagnostics.
segmentation_rows=[{'case':name,**r['diagnostics']} for name,r in case_results.
    items()]
display(segmentation_rows)

# Lightweight consistency check against the compact data-validation cache.
validation_cache_path=OUTPUT_DIR/'data_validation_derived.pt';
    validation_consistency=[]
if validation_cache_path.is_file():
    try: validation_cache=torch.
        load(validation_cache_path,map_location='cpu',weights_only=False)
    except TypeError: validation_cache=torch.
        load(validation_cache_path,map_location='cpu')
for name,r in case_results.items():
    if name not in validation_cache.get('derived', {}): continue
    cached=validation_cache['derived'][name]; expected_approaches=0;
    expected_responding=0
    for segment in cached['distance_segments']:
        expected_approaches += len(pa.
        detect_approach_events(segment['d_norm'],threshold=APPROACH_DISTANCE_THRESHOLD,closing_lag=
        for segment in cached['response_segments']:
            onsets=pa.
            detect_approach_events(segment['d_norm'],threshold=APPROACH_DISTANCE_THRESHOLD,closing_lag=
            for onset in onsets.tolist():

```

```

        stop=min(onset+RESPONSE_WINDOW_STEPS+1,len(segment['response']))
        expected_responding += int(onset<stop and bool((segment['response'][onset:
↪stop]>RESPONSE_THRESHOLD).any()))
        expected={'valid_frames':
↪validation_cache['coverage'][name]['valid_predator_prey_frames'],'temporal_segments':
↪len(cached['distance_segments']),'response_capable_segments':
↪len(cached['response_segments']),'approach_events':
↪expected_approaches,'responding_events':expected_responding}
        observed=r['diagnostics']; validation_consistency.append({'case':
↪name,**{f'validation_{k}':v for k,v in expected.items()},**{f'analysis_{k}':
↪observed[k] for k in expected},'all_match':all(observed[k]==v for k,v in_
↪expected.items())})
display(validation_consistency)

# Saving is optional because all results remain available in memory.
if SAVE_RESULTS:
    OUTPUT_DIR.mkdir(parents=True,exist_ok=True); torch.save(results,OUTPUT_DIR/
↪'predator_prey_metrics.pt')

```

Analyzing cases: 100%| | 7/7 [00:36<00:00, 5.23s/it]

```

[{'case': 'couzin_expert_16',
  'valid_frames': 20000,
  'invalid_frames': 0,
  'frame_clips': 20,
  'frame_level_segments': 20,
  'temporal_segments': 20,
  'response_segments': 20,
  'response_capable_segments': 20,
  'approach_events': 911,
  'response_approach_events': 911,
  'responding_events': 909,
  'total_approach_events': 911,
  'valid_propagation_events': 909,
  'fraction_valid_propagation_events': 0.9978046103183315},
{'case': 'couzin_expert_32',
  'valid_frames': 20000,
  'invalid_frames': 0,
  'frame_clips': 20,
  'frame_level_segments': 20,
  'temporal_segments': 20,
  'response_segments': 20,
  'response_capable_segments': 20,
  'approach_events': 1117,
  'response_approach_events': 1117,
  'responding_events': 1117,
  'total_approach_events': 1117,
  'valid_propagation_events': 1117,

```

```

    'fraction_valid_propagation_events': 1.0},
{'case': 'couzin_imitation_16',
 'valid_frames': 20000,
 'invalid_frames': 0,
 'frame_clips': 20,
 'frame_level_segments': 20,
 'temporal_segments': 20,
 'response_segments': 20,
 'response_capable_segments': 20,
 'approach_events': 2685,
 'response_approach_events': 2685,
 'responding_events': 276,
 'total_approach_events': 2685,
 'valid_propagation_events': 11,
 'fraction_valid_propagation_events': 0.00409683426443203},
{'case': 'couzin_imitation_32',
 'valid_frames': 20000,
 'invalid_frames': 0,
 'frame_clips': 20,
 'frame_level_segments': 20,
 'temporal_segments': 20,
 'response_segments': 20,
 'response_capable_segments': 20,
 'approach_events': 3165,
 'response_approach_events': 3165,
 'responding_events': 321,
 'total_approach_events': 3165,
 'valid_propagation_events': 26,
 'fraction_valid_propagation_events': 0.008214849921011059},
{'case': 'bio_expert_32',
 'valid_frames': 11879,
 'invalid_frames': 0,
 'frame_clips': 35,
 'frame_level_segments': 6917,
 'temporal_segments': 6917,
 'response_segments': 8681,
 'response_capable_segments': 1915,
 'approach_events': 159,
 'response_approach_events': 77,
 'responding_events': 26,
 'total_approach_events': 77,
 'valid_propagation_events': 14,
 'fraction_valid_propagation_events': 0.18181818181818182},
{'case': 'bio_imitation_16',
 'valid_frames': 20000,
 'invalid_frames': 0,
 'frame_clips': 20,
 'frame_level_segments': 20,

```

```

'temporal_segments': 20,
'response_segments': 20,
'response_capable_segments': 20,
'approach_events': 502,
'response_approach_events': 502,
'responding_events': 501,
'total_approach_events': 502,
'valid_propagation_events': 501,
'fraction_valid_propagation_events': 0.99800796812749},
{'case': 'bio_imitation_32',
 'valid_frames': 20000,
 'invalid_frames': 0,
 'frame_clips': 20,
 'frame_level_segments': 20,
 'temporal_segments': 20,
 'response_segments': 20,
 'response_capable_segments': 20,
 'approach_events': 719,
 'response_approach_events': 719,
 'responding_events': 719,
 'total_approach_events': 719,
 'valid_propagation_events': 719,
 'fraction_valid_propagation_events': 1.0}]

[{'case': 'couzin_expert_16',
 'validation_valid_frames': 20000,
 'validation_temporal_segments': 20,
 'validation_response_capable_segments': 20,
 'validation_approach_events': 911,
 'validation_responding_events': 909,
 'analysis_valid_frames': 20000,
 'analysis_temporal_segments': 20,
 'analysis_response_capable_segments': 20,
 'analysis_approach_events': 911,
 'analysis_responding_events': 909,
 'all_match': True},
 {'case': 'couzin_expert_32',
 'validation_valid_frames': 20000,
 'validation_temporal_segments': 20,
 'validation_response_capable_segments': 20,
 'validation_approach_events': 1117,
 'validation_responding_events': 1117,
 'analysis_valid_frames': 20000,
 'analysis_temporal_segments': 20,
 'analysis_response_capable_segments': 20,
 'analysis_approach_events': 1117,
 'analysis_responding_events': 1117,
 'all_match': True},

```

```

{'case': 'couzin_imitation_16',
 'validation_valid_frames': 20000,
 'validation_temporal_segments': 20,
 'validation_response_capable_segments': 20,
 'validation_approach_events': 2685,
 'validation_responding_events': 276,
 'analysis_valid_frames': 20000,
 'analysis_temporal_segments': 20,
 'analysis_response_capable_segments': 20,
 'analysis_approach_events': 2685,
 'analysis_responding_events': 276,
 'all_match': True},
{'case': 'couzin_imitation_32',
 'validation_valid_frames': 20000,
 'validation_temporal_segments': 20,
 'validation_response_capable_segments': 20,
 'validation_approach_events': 3165,
 'validation_responding_events': 321,
 'analysis_valid_frames': 20000,
 'analysis_temporal_segments': 20,
 'analysis_response_capable_segments': 20,
 'analysis_approach_events': 3165,
 'analysis_responding_events': 321,
 'all_match': True},
{'case': 'bio_expert_32',
 'validation_valid_frames': 11879,
 'validation_temporal_segments': 6917,
 'validation_response_capable_segments': 1915,
 'validation_approach_events': 159,
 'validation_responding_events': 26,
 'analysis_valid_frames': 11879,
 'analysis_temporal_segments': 6917,
 'analysis_response_capable_segments': 1915,
 'analysis_approach_events': 159,
 'analysis_responding_events': 26,
 'all_match': True},
{'case': 'bio_imitation_16',
 'validation_valid_frames': 20000,
 'validation_temporal_segments': 20,
 'validation_response_capable_segments': 20,
 'validation_approach_events': 502,
 'validation_responding_events': 501,
 'analysis_valid_frames': 20000,
 'analysis_temporal_segments': 20,
 'analysis_response_capable_segments': 20,
 'analysis_approach_events': 502,
 'analysis_responding_events': 501,
 'all_match': True},

```

```
{'case': 'bio_imitation_32',
 'validation_valid_frames': 20000,
 'validation_temporal_segments': 20,
 'validation_response_capable_segments': 20,
 'validation_approach_events': 719,
 'validation_responding_events': 719,
 'analysis_valid_frames': 20000,
 'analysis_temporal_segments': 20,
 'analysis_response_capable_segments': 20,
 'analysis_approach_events': 719,
 'analysis_responding_events': 719,
 'all_match': True}]
```

```
[5]: # Shared visual language: source panels, expert/imitation colors, and
      ↪ group-size shades.
COLORS={'expert_16': '#1f77b4', 'expert_32': '#0b3c6f', 'imitation_16':
      ↪ '#ff8c42', 'imitation_32': '#c44e00'}
def _style(name):
    return ('cousin' if name.startswith('cousin') else 'biological'), next(x for x
    ↪ in COLORS if x in name)

# Two-panel helper for all risk-conditioned curve metrics.
def plot_curve(metric, ylabel, xlabel='Normalized predator-to-nearest-prey
    ↪ distance'):
    fig, axes = plt.subplots(1, 2, figsize=(12, 4), sharey=True)
    for ax, source in zip(axes, ('cousin', 'biological')):
        for name, r in case_results.items():
            src, style = _style(name)
            if src != source: continue
            c = r['curves'][metric]; x, y = c['bin_center'].cpu(), c['mean'].cpu(); ax.
            ↪ plot(x, y, marker='o', ms=3, color=COLORS[style], label=style.replace('_', ' ').
            ↪ title()); ax.fill_between(x, c['ci_low'].cpu(), c['ci_high'].
            ↪ cpu(), color=COLORS[style], alpha=.15)
            ax.set(title=source.
            ↪ title(), xlabel=xlabel, ylabel=ylabel, xlim=RISK_RANGES[source]); ax.
            ↪ grid(alpha=.25); handles, labels = ax.get_legend_handles_labels(); ax.
            ↪ legend(handles, labels, frameon=False) if handles else None
    fig.tight_layout(); plt.show()

# Distribution helper for per-clip scalars or per-event observations.
def plot_scalar(metric, ylabel, event=False):
    names, data = [], []
    for name, r in case_results.items():
        v = r['events'].get(metric) if event else r['per_clip'].get(metric)
        if v is not None and v.numel(): names.append(name.replace('_', '\n')); data.
        ↪ append(v.cpu().numpy())
    if data:
```

```

fig,ax=plt.subplots(figsize=(max(8,1.2*len(data)),4)); ax.
↳boxplot(data,tick_labels=names,showfliers=False); ax.set_ylabel(ylabel); ax.
↳grid(axis='y',alpha=.25); fig.tight_layout(); plt.show()

# Event-relative helper for NND and polarization changes after approach onset.
def plot_event(key,ylabel):
    fig,axes=plt.subplots(1,2,figsize=(12,4),sharey=True)
    for ax,source in zip(axes,('couzin','biological')):
        for name,r in case_results.items():
            src,style=_style(name); valid=[x for x in r[key] if x.numel()]
            if src==source and valid:
                v=torch.cat(valid).cpu(); ax.plot(torch.arange(v.shape[1]),torch.
↳nanmean(v,0),color=COLORS[style],label=style.replace('_', ' ').title())
                ax.axhline(0,color='.4',lw=.8); ax.set(title=source.title(),xlabel='Steps_
↳since approach onset',ylabel=ylabel); ax.grid(alpha=.25); handles,labels=ax.
↳get_legend_handles_labels(); ax.legend(handles,labels,frameon=False) if
↳handles else None
    fig.tight_layout(); plt.show()

# Whole-sequence helper for DoS and DoA time evolution.
def plot_timeline(key,ylabel):
    fig,axes=plt.subplots(1,2,figsize=(12,4),sharey=True)
    for ax,source in zip(axes,('couzin','biological')):
        for name,r in case_results.items():
            src,style=_style(name); series=r[key]; length=max((len(x) for x in
↳series),default=0)
            if src==source and length:
                padded=torch.full((len(series),length),torch.nan,device=series[0].
↳device,dtype=series[0].dtype)
                for i,x in enumerate(series): padded[i,:len(x)]=x
                ax.plot(torch.nanmean(padded,0).cpu(),color=COLORS[style],label=style.
↳replace('_', ' ').title())
                ax.set(title=source.title(),xlabel='Source step',ylabel=ylabel); ax.
↳grid(alpha=.25); handles,labels=ax.get_legend_handles_labels(); ax.
↳legend(handles,labels,frameon=False) if handles else None
    fig.tight_layout(); plt.show()

# Numerical outputs mirror every plot and remain available for later use.
numerical_outputs = {}

# Lightweight HTML-table formatting avoids a separate pandas pipeline.
def _format_table_value(value):
    if torch.is_tensor(value) and value.numel()==1: value=value.item()
    if isinstance(value,(float,np.floating)):
        return 'NaN' if not np.isfinite(value) else f'{value:.8g}'
    return str(value)

```

```

def display_table(rows,title,max_height=420):
    if not rows:
        display(HTML(f'<h4>{html.escape(title)}</h4><em>No valid values.</em>'));
        return
    columns=list(rows[0]); head=''.join(f'<th>{html.escape(str(c))}</th>' for c in
    columns)
    body=''.join('<tr>'+''.join(f'<td>{html.escape(_format_table_value(row.
    get(c,""))}</td>' for c in columns)+'</tr>' for row in rows)
    style='border-collapse:collapse;width:100%;font-size:12px;text-align:right'
    display(HTML(f'<style>.metric-table th,.metric-table td{{padding:5px 9px;
    border-bottom:1px solid #eee;white-space:nowrap}}.metric-table
    th{{text-align:right}}</style><h4>{html.escape(title)}</h4><div
    style="max-height:{max_height}px;overflow:auto;border:1px solid #ddd"><table
    class="metric-table" style="{style}"><thead style="position:sticky;top:0;
    background:white"><tr>{head}</tr></thead><tbody>{body}</tbody></table></
    div>'))

# Table builders for curves, distributions, event profiles, and timelines.
def show_curve_values(metric):
    out,rows={},[]
    for name,r in case_results.items():
        c={k:r['curves'][metric][k].detach().cpu().tolist() for k in
        ('bin_center','mean','ci_low','ci_high','count','cluster_count','supported')};
        out[name]=c
        rows.extend({'case':name,'bin':i,'distance':c['bin_center'][i],'mean':
        c['mean'][i],'ci_low':c['ci_low'][i],'ci_high':c['ci_high'][i],'n':
        c['count'][i],'clusters':c['cluster_count'][i],'supported':
        c['supported'][i]} for i in range(len(c['bin_center'])))
    numerical_outputs[metric]=out; display_table(rows,f'{metric}: risk-conditioned
    values'); return out
def show_distribution_values(metric,event=False):
    full,summary,rows={},{},{},[]
    for name,r in case_results.items():
        v=r['events'].get(metric) if event else r['per_clip'].get(metric)
        if v is None: continue
        v=v.detach().float().cpu(); v=v[torch.isfinite(v)]; full[name]=v
        valid_clips=int(torch.isfinite(r['per_clip'][metric]).sum()) if metric in
        r['per_clip'] else np.nan
        s={'case':name,'n':len(v),'valid_clips':valid_clips,'mean':float(v.mean()) if
        len(v) else np.nan,'std':float(v.std(unbiased=False)) if len(v) else np.
        nan,'median':float(v.median()) if len(v) else np.nan,'q25':float(torch.
        quantile(v,.25)) if len(v) else np.nan,'q75':float(torch.quantile(v,.75)) if
        len(v) else np.nan,'min':float(v.min()) if len(v) else np.nan,'max':float(v.
        max()) if len(v) else np.nan}; summary[name]={k:x for k,x in s.items() if k!
        ='case'}; rows.append(s)

```

```

numerical_outputs[metric]={'summary':summary,'values':full};
    ↳display_table(rows,f'{metric}: numerical summary'); return summary
def show_event_profile_values(key):
    out,rows={},[]
    for name,r in case_results.items():
        valid=[x.detach().cpu() for x in r[key] if x.numel()]
        if not valid: continue
        v=torch.cat(valid); mean=torch.nanmean(v,0); std=torch.nanmean((v-mean)**2,0).
        ↳sqrt(); count=torch.isfinite(v).sum(0); out[name]={'step':list(range(v.
        ↳shape[1])), 'mean':mean.tolist(), 'std':std.tolist(), 'count':count.tolist()}
        rows.extend({'case':name, 'step':step, 'mean':mean[step].item(), 'std':std[step].
        ↳item(), 'n':count[step].item()} for step in range(v.shape[1]))
    numerical_outputs[key]=out; display_table(rows,f'{key}: event-relative
    ↳values'); return out
def show_timeline_values(key,aggregate_metric):
    full,summary,summary_rows,timeline_rows={},{},{},[]
    for name,r in case_results.items():
        length=max((len(x) for x in r[key]),default=0)
        if not length: continue
        timeline=torch.full((len(r[key]),length),torch.nan,device=r[key][0].
        ↳device, dtype=r[key][0].dtype)
        for i,x in enumerate(r[key]): timeline[i,:len(x)]=x
        timeline=timeline.detach().cpu(); mean_timeline=torch.nanmean(timeline,0);
        ↳timeline_count=torch.isfinite(timeline).sum(0);
        ↳aggregate=r['per_clip'][aggregate_metric].detach().cpu();
        ↳full[name]={'mean_timeline':mean_timeline, 'timeline_count':
        ↳timeline_count, 'per_clip_aggregate':aggregate}
        s={'case':name, 'steps':length, 'clips':len(timeline), 'aggregate_mean':
        ↳float(aggregate.mean()), 'aggregate_std':float(aggregate.
        ↳std(unbiased=False))}; summary[name]={k:x for k,x in s.items() if k!='case'};
        ↳summary_rows.append(s)
        timeline_rows.extend({'case':name, 'step':step, 'mean':mean_timeline[step].
        ↳item(), 'clips':timeline_count[step].item()} for step in range(length))
    numerical_outputs[aggregate_metric]={'summary':summary,'values':full};
    ↳display_table(summary_rows,f'{aggregate_metric}: aggregate values');
    ↳display_table(timeline_rows,f'{aggregate_metric}: mean time evolution');
    ↳return summary

```

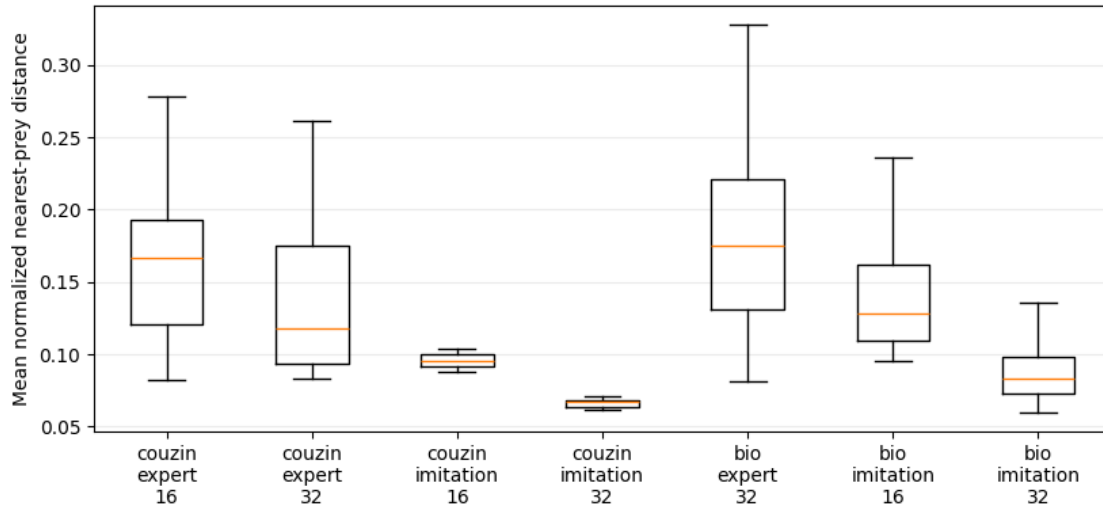
2 Predator Response Metrics

These metrics describe how the predator is positioned and oriented relative to the nearest-prey proxy. Distance and the primary three-step closing speed are normalized by the source-specific arena diagonal, so source families remain comparable.

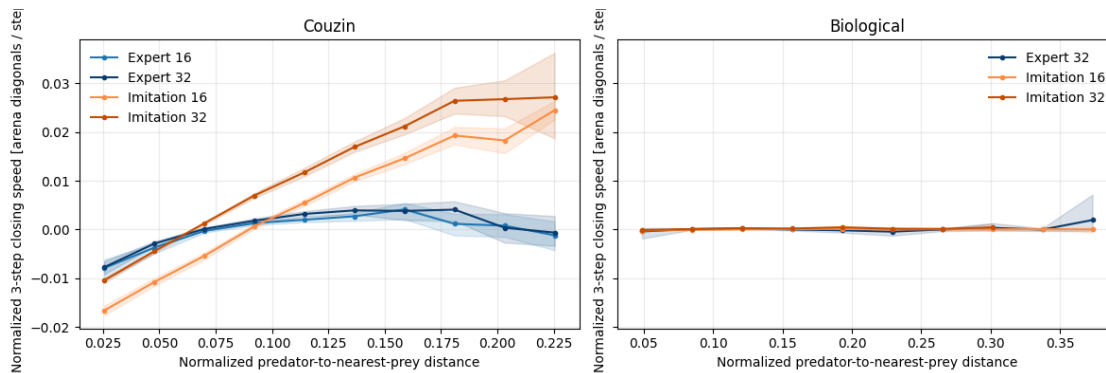
Each plot is followed by its corresponding numerical table.

```
[6]: # Overall normalized proximity to the nearest-prey proxy.
plot_scalar('predator_distance', 'Mean normalized nearest-prey distance');
↳ show_distribution_values('predator_distance')

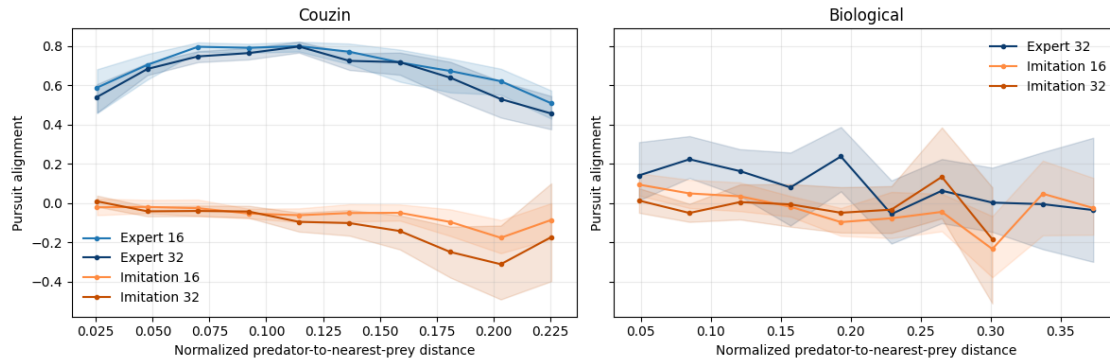
# Distance change and heading alignment conditioned on the common risk axis.
plot_curve('closing_speed', 'Normalized 3-step closing speed [arena diagonals / step]');
↳ show_curve_values('closing_speed')
plot_curve('pursuit_alignment', 'Pursuit alignment');
↳ show_curve_values('pursuit_alignment')
```



<IPython.core.display.HTML object>



<IPython.core.display.HTML object>



<IPython.core.display.HTML object>

```
[6]: {'couzin_expert_16': {'bin_center': [0.025510767474770546,
0.04772769659757614,
0.06994462013244629,
0.09216155111789703,
0.11437846720218658,
0.13659539818763733,
0.15881231427192688,
0.18102926015853882,
0.20324617624282837,
0.22546309232711792],
'mean': [0.588869035243988,
0.7055052518844604,
0.79722660779953,
0.7920950651168823,
0.8003886342048645,
0.7718997597694397,
0.7172713875770569,
0.6736328601837158,
0.6216883659362793,
0.5089919567108154],
'ci_low': [0.45869314670562744,
0.6301241517066956,
0.7714666128158569,
0.7664241790771484,
0.7729427218437195,
0.7085618376731873,
0.6192129254341125,
0.564422070980072,
0.5525341033935547,
0.43061038851737976],
'ci_high': [0.6825283765792847,
0.7603166103363037,
```

```

0.8194558024406433,
0.8123061656951904,
0.8223829865455627,
0.8133172988891602,
0.7825085520744324,
0.7364869713783264,
0.6852999329566956,
0.5742287635803223],
'count': [1354, 2337, 3516, 2377, 1924, 1191, 606, 445, 406, 265],
'cluster_count': [20, 20, 20, 20, 20, 20, 20, 20, 20, 20],
'supported': [True, True, True, True, True, True, True, True, True, True]},
'couzin_expert_32': {'bin_center': [0.025510767474770546,
0.04772769659757614,
0.06994462013244629,
0.09216155111789703,
0.11437846720218658,
0.13659539818763733,
0.15881231427192688,
0.18102926015853882,
0.20324617624282837,
0.22546309232711792],
'mean': [0.5412760376930237,
0.6843693256378174,
0.7481709718704224,
0.7649176716804504,
0.7984341382980347,
0.7254133224487305,
0.7188200950622559,
0.6395301818847656,
0.5305503010749817,
0.4568876624107361],
'ci_low': [0.4611952006816864,
0.6530537009239197,
0.7186441421508789,
0.7323272228240967,
0.7672175168991089,
0.6789385080337524,
0.6543323397636414,
0.538141667842865,
0.4360884726047516,
0.37630924582481384],
'ci_high': [0.6068852543830872,
0.7144615054130554,
0.7746469974517822,
0.7909378409385681,
0.8231647610664368,
0.7638795375823975,

```

```

0.7674732208251953,
0.720162034034729,
0.6209153532981873,
0.5456839799880981],
'count': [2034, 3036, 3230, 2585, 1883, 1043, 712, 426, 358, 309],
'cluster_count': [20, 20, 20, 20, 20, 20, 20, 20, 20, 20],
'supported': [True, True, True, True, True, True, True, True, True, True]},
'couzin_imitation_16': {'bin_center': [0.025510767474770546,
0.04772769659757614,
0.06994462013244629,
0.09216155111789703,
0.11437846720218658,
0.13659539818763733,
0.15881231427192688,
0.18102926015853882,
0.20324617624282837,
0.22546309232711792],
'mean': [-0.02099907025694847,
-0.0192317645996809,
-0.0257592611014843,
-0.05288240686058998,
-0.06145802140235901,
-0.050430476665496826,
-0.04929546266794205,
-0.09599476307630539,
-0.17650915682315826,
-0.08567522466182709],
'ci_low': [-0.06121399998664856,
-0.05098720267415047,
-0.06544353812932968,
-0.0772600993514061,
-0.09691590815782547,
-0.09386009722948074,
-0.08913753926753998,
-0.16763006150722504,
-0.25554606318473816,
-0.1833461970090866],
'ci_high': [0.022560400888323784,
0.01641254313290119,
0.017982983961701393,
-0.027940090745687485,
-0.026702240109443665,
-0.005433287471532822,
-0.006719348952174187,
-0.031595055013895035,
-0.08649314194917679,
-0.00023489286832045764],

```

```

'count': [2057, 3081, 3505, 3178, 2674, 2041, 1294, 733, 468, 216],
'cluster_count': [20, 20, 20, 20, 20, 20, 20, 20, 20, 20],
'supported': [True, True, True, True, True, True, True, True, True, True]},
'couzin_imitation_32': {'bin_center': [0.025510767474770546,
0.04772769659757614,
0.06994462013244629,
0.09216155111789703,
0.11437846720218658,
0.13659539818763733,
0.15881231427192688,
0.18102926015853882,
0.20324617624282837,
0.22546309232711792],
'mean': [0.009323344565927982,
-0.041791390627622604,
-0.03990786522626877,
-0.042599473148584366,
-0.09529382735490799,
-0.10167519003152847,
-0.14217416942119598,
-0.24914610385894775,
-0.31134936213493347,
-0.17316126823425293],
'ci_low': [-0.018059320747852325,
-0.06708267331123352,
-0.06606120616197586,
-0.07443977892398834,
-0.14514745771884918,
-0.16599303483963013,
-0.23618370294570923,
-0.3796880543231964,
-0.4908537268638611,
-0.39807629585266113],
'ci_high': [0.038322340697050095,
-0.01686752960085869,
-0.012375419028103352,
-0.013239649124443531,
-0.04863104969263077,
-0.034884013235569,
-0.06265123188495636,
-0.11987054347991943,
-0.11428319662809372,
0.1007751077413559],
'count': [3669, 5020, 4557, 2875, 1615, 807, 349, 154, 66, 37],
'cluster_count': [20, 20, 20, 20, 20, 20, 20, 20, 19, 16],
'supported': [True, True, True, True, True, True, True, True, True, True]},
'bio_expert_32': {'bin_center': [0.04882354661822319,

```

```

0.08488023281097412,
0.12093693017959595,
0.15699362754821777,
0.1930502951145172,
0.22910699248313904,
0.26516368985176086,
0.3012203872203827,
0.33727705478668213,
0.37333375215530396],
'mean': [0.14136667549610138,
0.223573699593544,
0.16339640319347382,
0.08014575392007828,
0.23804539442062378,
-0.05592425540089607,
0.06327660381793976,
0.0027759724762290716,
-0.005131789483129978,
-0.03530222922563553],
'ci_low': [0.01430025789886713,
0.12794692814350128,
0.03557378053665161,
-0.11440131068229675,
0.0597492977976799,
-0.20636482536792755,
-0.1020302027463913,
-0.14684545993804932,
-0.23478184640407562,
-0.2995177209377289],
'ci_high': [0.3110983967781067,
0.34167906641960144,
0.27498486638069153,
0.25791287422180176,
0.38824278116226196,
0.11544506251811981,
0.22488164901733398,
0.18062129616737366,
0.25996139645576477,
0.33337241411209106],
'count': [1175, 1989, 1834, 1358, 1101, 1038, 829, 533, 548, 562],
'cluster_count': [28, 31, 31, 29, 31, 28, 29, 23, 20, 17],
'supported': [True, True, True, True, True, True, True, True, True, True]},
'bio_imitation_16': {'bin_center': [0.04882354661822319,
0.08488023281097412,
0.12093693017959595,
0.15699362754821777,
0.1930502951145172,

```

```

0.22910699248313904,
0.26516368985176086,
0.3012203872203827,
0.33727705478668213,
0.37333375215530396],
'mean': [0.0943816602230072,
0.04956193268299103,
0.03488432988524437,
-0.01811004802584648,
-0.0966467410326004,
-0.07724039256572723,
-0.04455046355724335,
-0.23381082713603973,
0.04801030829548836,
-0.024784468114376068],
'ci_low': [0.0384289026260376,
-0.02513851784169674,
-0.0408160425722599,
-0.0687626451253891,
-0.1669626235961914,
-0.17824822664260864,
-0.14344634115695953,
-0.3782467246055603,
-0.16439829766750336,
-0.1610119789838791],
'ci_high': [0.1519109159708023,
0.11913572996854782,
0.10557916015386581,
0.03602856397628784,
-0.024961844086647034,
0.057175811380147934,
0.0476890429854393,
-0.06581317633390427,
0.2164064347743988,
0.12884874641895294],
'count': [3373, 3545, 3534, 2494, 1761, 1223, 893, 720, 499, 208],
'cluster_count': [20, 20, 20, 20, 20, 19, 16, 14, 12, 8],
'supported': [True, True, True, True, True, True, True, True, True, True]},
'bio_imitation_32': {'bin_center': [0.04882354661822319,
0.08488023281097412,
0.12093693017959595,
0.15699362754821777,
0.1930502951145172,
0.22910699248313904,
0.26516368985176086,
0.3012203872203827,
0.33727705478668213,

```

```

0.37333375215530396],
'mean': [0.013158153742551804,
-0.0501137338578701,
0.005399176850914955,
-0.006103808991611004,
-0.0489531084895134,
-0.0331835113465786,
0.13386845588684082,
-0.1839841902256012,
nan,
nan],
'ci_low': [-0.04922095313668251,
-0.0957447737455368,
-0.08331044763326645,
-0.12021404504776001,
-0.1500394344329834,
-0.15183834731578827,
-0.07762745022773743,
-0.5116670727729797,
nan,
nan],
'ci_high': [0.07577858120203018,
-0.003639424219727516,
0.09524598717689514,
0.09709873795509338,
0.08119729906320572,
0.08658770471811295,
0.3859427273273468,
0.08073925971984863,
nan,
nan],
'count': [5941, 4960, 2949, 1366, 790, 434, 329, 157, 101, 53],
'cluster_count': [20, 20, 20, 20, 17, 13, 9, 7, 3, 2],
'supported': [True, True, True, True, True, True, True, True, False, False]]}

```

3 Prey Response Metrics

Continuous response fixes the predator-away direction at t . Valid responses satisfy $R > 0.10$ within the next 20 available steps; latency is in source steps, and truncated windows are represented explicitly by availability masks rather than silently dropped.

Escape alignment describes the instantaneous direction of motion. Reaction latency and normalized reaction distance include only responders, while response fraction reports how much of the group responded; the compact sensitivity table checks thresholds 0.075, 0.10, and 0.125.

[7]: *# Directional escape behavior and the continuous change in away-alignment.*

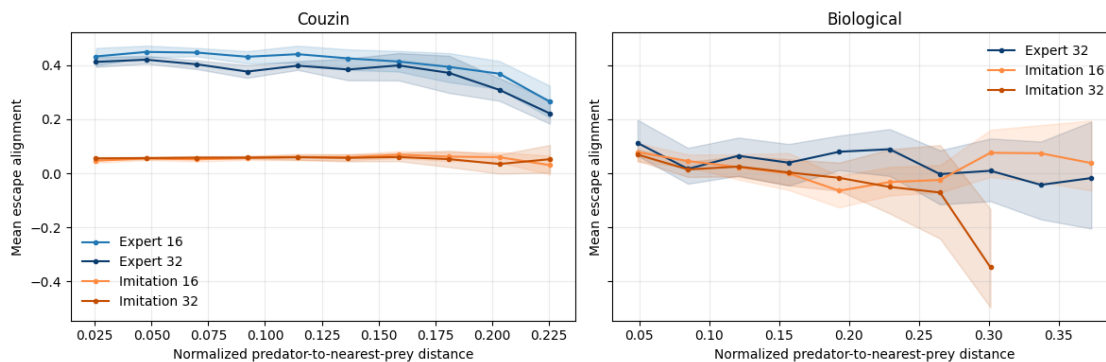
```

plot_curve('escape_alignment', 'Mean escape alignment');
↳ show_curve_values('escape_alignment')
plot_curve('continuous_predator_response_curve', 'Mean continuous response
↳ R', 'Normalized predator-to-prey distance');
↳ show_curve_values('continuous_predator_response_curve')

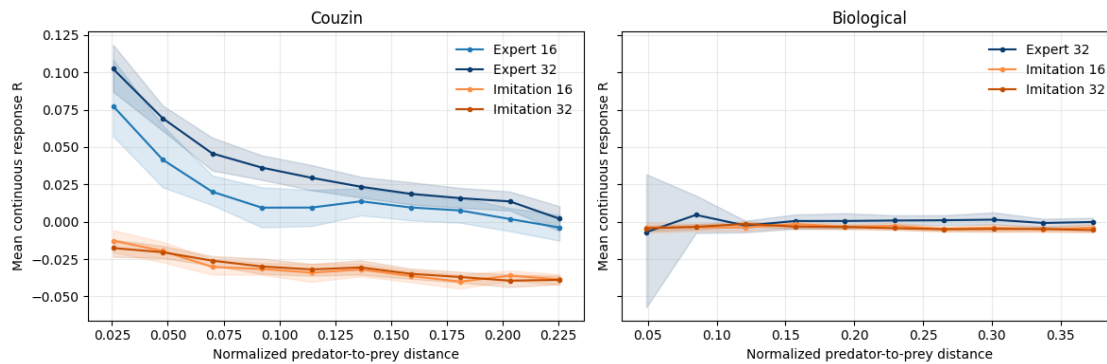
# Event-level timing, spatial response point, and fraction of responding prey.
plot_scalar('reaction_latency', 'Reaction latency [steps]', event=True);
↳ show_distribution_values('reaction_latency', event=True)
plot_scalar('reaction_distance_norm', 'Normalized predator-to-responding-prey
↳ distance', event=True);
↳ show_distribution_values('reaction_distance_norm', event=True)
plot_scalar('response_fraction', 'Response fraction', event=True);
↳ show_distribution_values('response_fraction', event=True)

# Compact threshold-sensitivity summary; the primary threshold remains 0.10.
sensitivity_rows=[]
for case_name, thresholds in results['response_threshold_sensitivity'].items():
    for threshold, summary in thresholds.items():
        sensitivity_rows.append({'case': case_name, 'threshold': threshold, 'events':
↳ summary['events'], 'contributing_clips':
↳ summary['contributing_clips'], 'mean_response_fraction':
↳ float(summary['mean_response_fraction']).detach().
↳ cpu(), 'fraction_events_two_plus': float(summary['fraction_events_two_plus']).
↳ detach().cpu()})
display_table(sensitivity_rows, 'Response-threshold sensitivity')

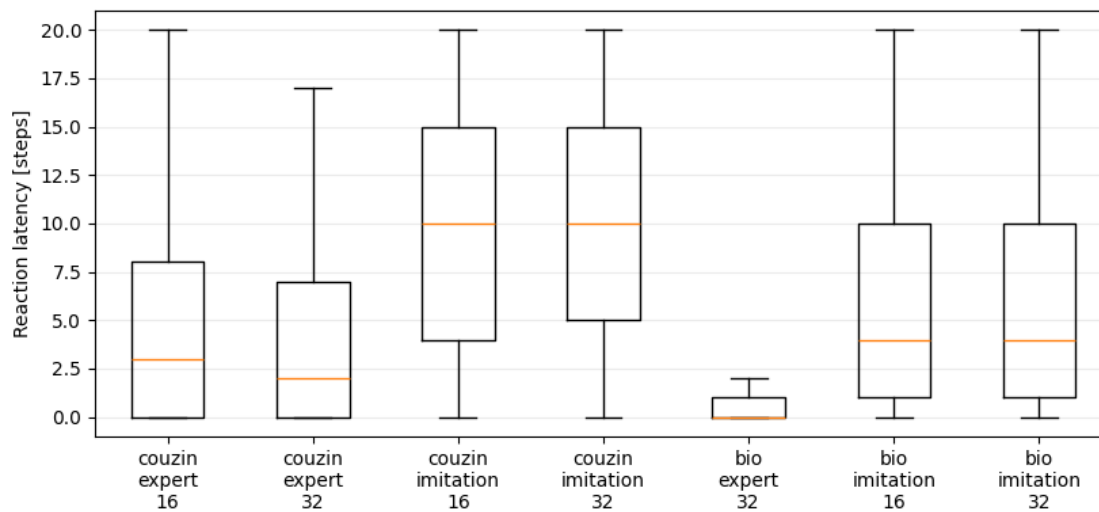
```



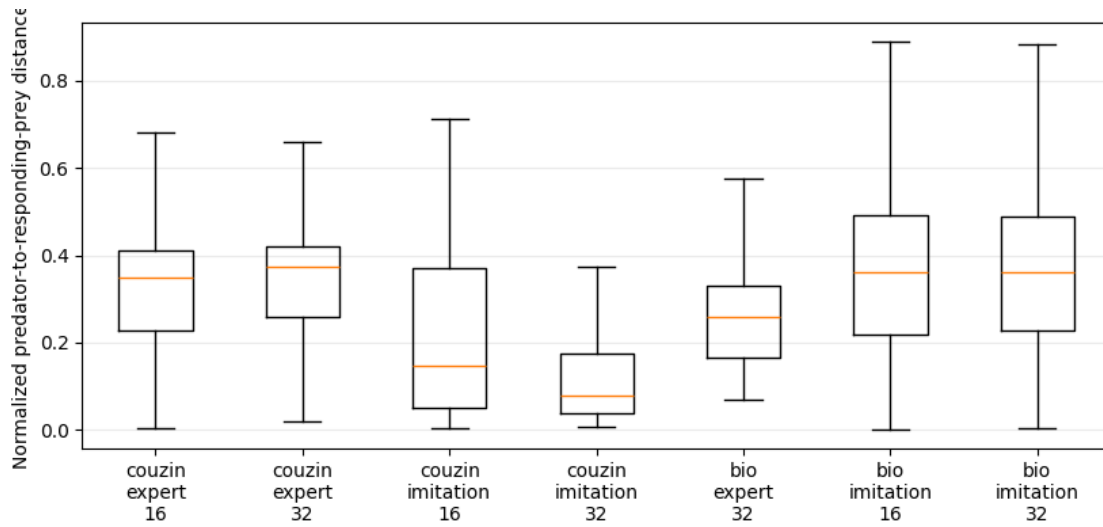
<IPython.core.display.HTML object>



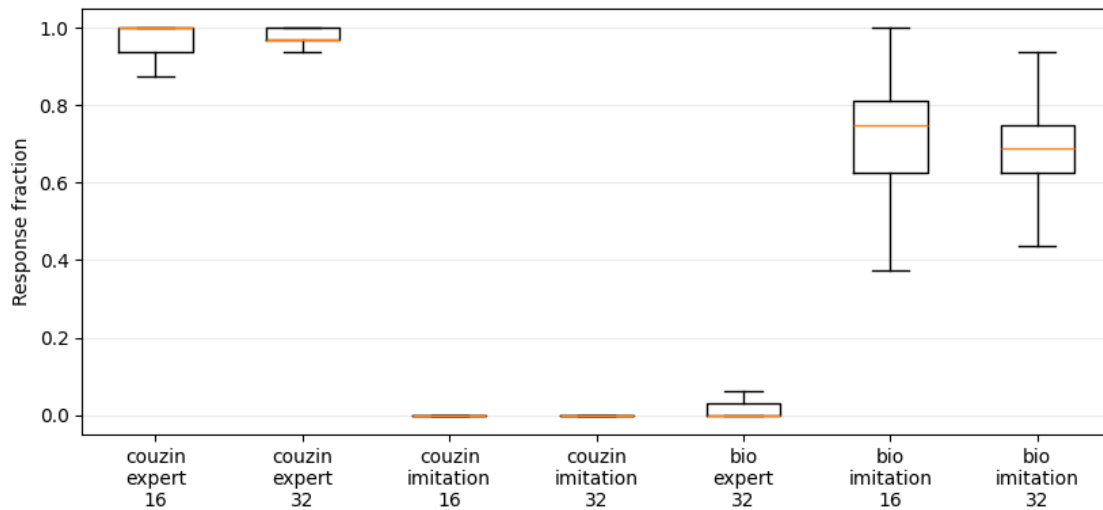
<IPython.core.display.HTML object>



<IPython.core.display.HTML object>



<IPython.core.display.HTML object>



<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

4 Collective Response Metrics

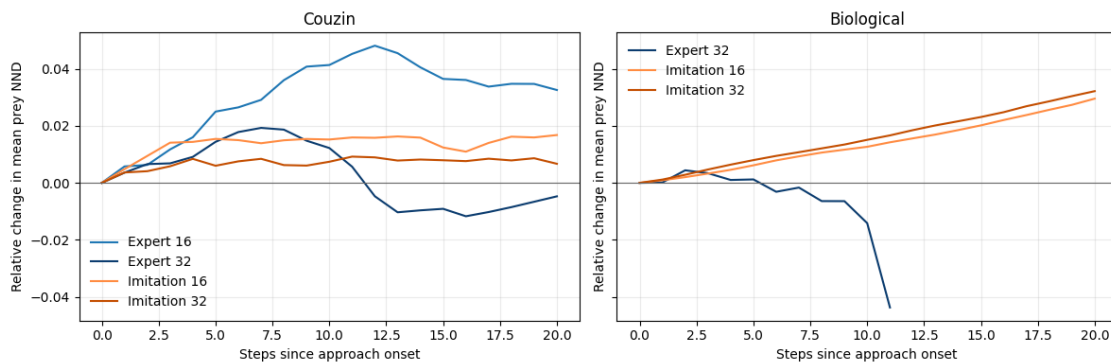
Event-relative NND reports the relative change from approach onset, while polarization reports the absolute change. DoS and DoA summarize all valid frames; cascade and propagation reuse the same individual first-response times, with propagation defined only when at least two prey respond.

The final table separates the 16-to-32 generalization shift from imitation fidelity at each group size.

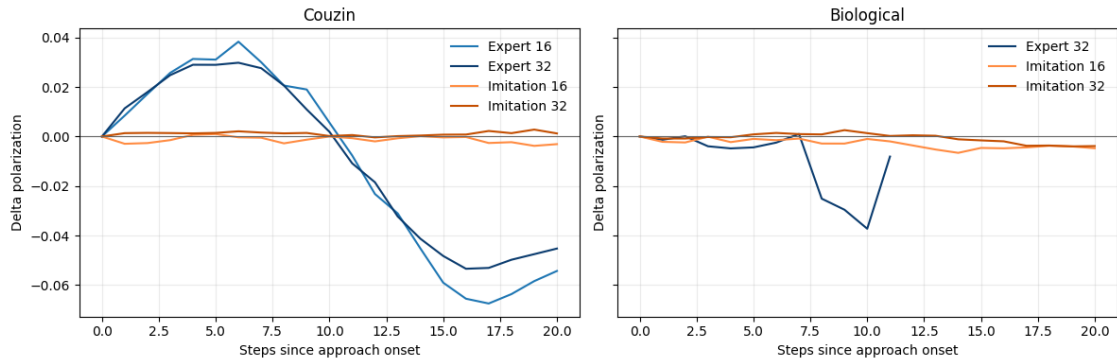
```
[8]: # Approach-relative changes in density and global alignment.
plot_event('nnd_during_approach','Relative change in mean prey NND');
    ↪show_event_profile_values('nnd_during_approach')
plot_event('polarization_during_approach','Delta polarization');
    ↪show_event_profile_values('polarization_during_approach')

# Whole-sequence local density and nearest-neighbor alignment.
plot_timeline('dos_timeline','Degree of Swarm');
    ↪show_timeline_values('dos_timeline','dos')
plot_timeline('doa_timeline','Degree of Alignment');
    ↪show_timeline_values('doa_timeline','doa')

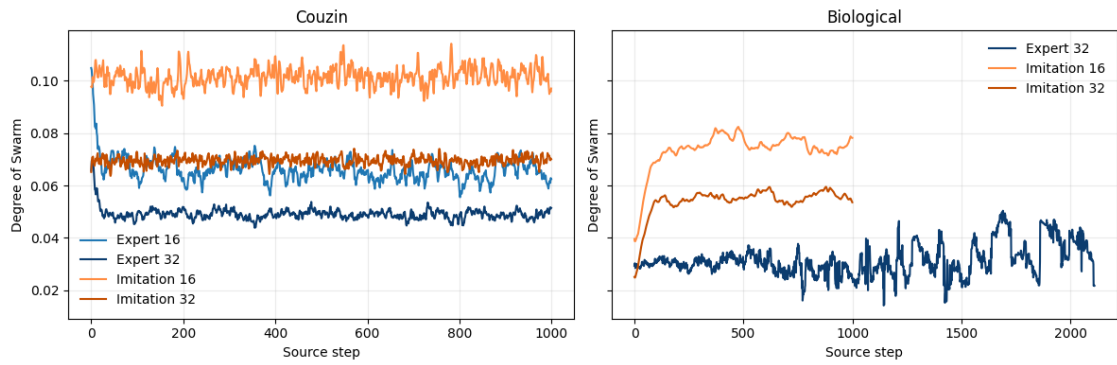
# Event-level response spread and temporal propagation through the swarm.
plot_scalar('cascade_size','Cascade size',event=True);
    ↪show_distribution_values('cascade_size',event=True)
plot_scalar('cascade_fraction','Cascade fraction',event=True);
    ↪show_distribution_values('cascade_fraction',event=True)
plot_scalar('propagation_time','Propagation time [steps]',event=True);
    ↪show_distribution_values('propagation_time',event=True)
plot_scalar('mean_propagation_delay','Mean propagation delay
    ↪[steps]',event=True);
    ↪show_distribution_values('mean_propagation_delay',event=True)
propagation_rows=[{'case':name,'valid_propagation_events':
    ↪r['diagnostics']['valid_propagation_events'],'total_approach_events':
    ↪r['diagnostics']['total_approach_events'],'fraction_valid':
    ↪r['diagnostics']['fraction_valid_propagation_events']}] for name,r in
    ↪case_results.items()]
display_table(propagation_rows,'Propagation support (at least two responders)')
```



<IPython.core.display.HTML object>

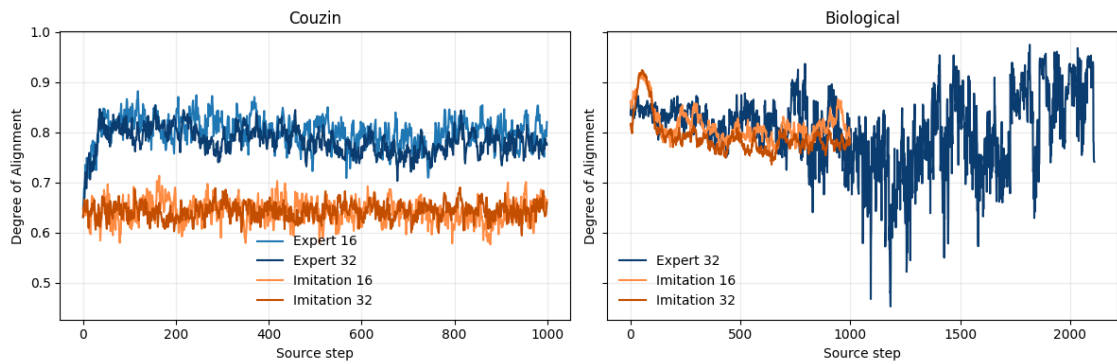


<IPython.core.display.HTML object>



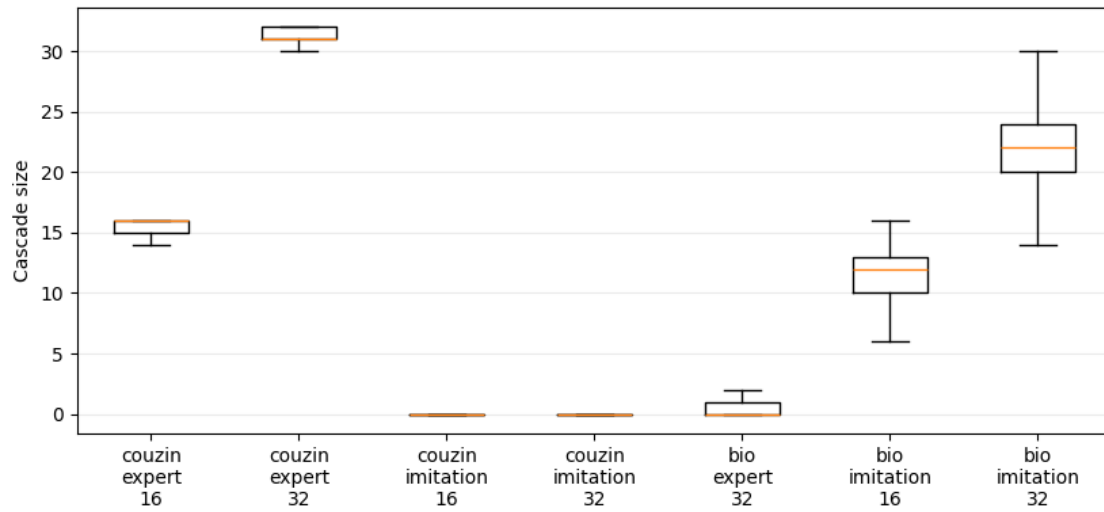
<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

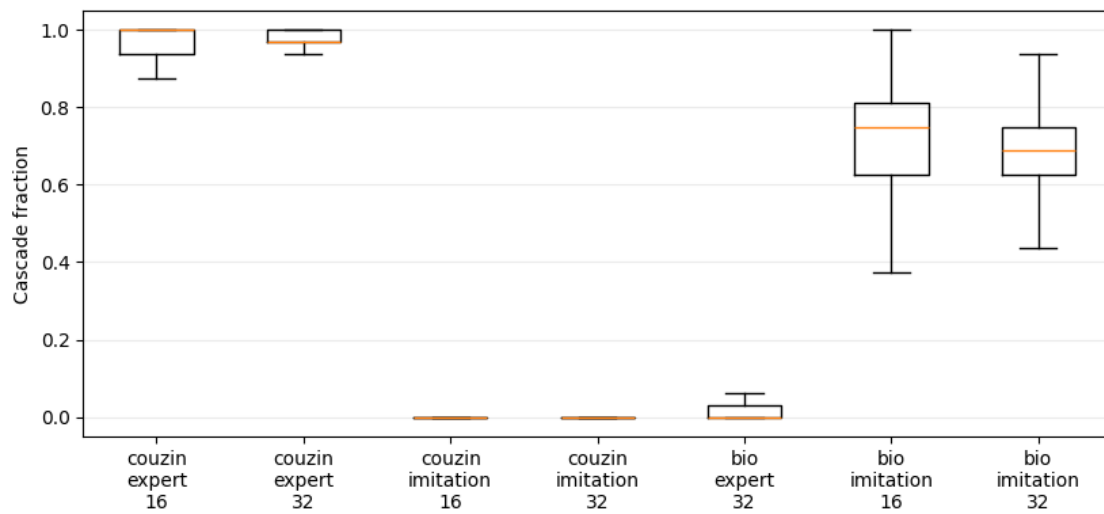


<IPython.core.display.HTML object>

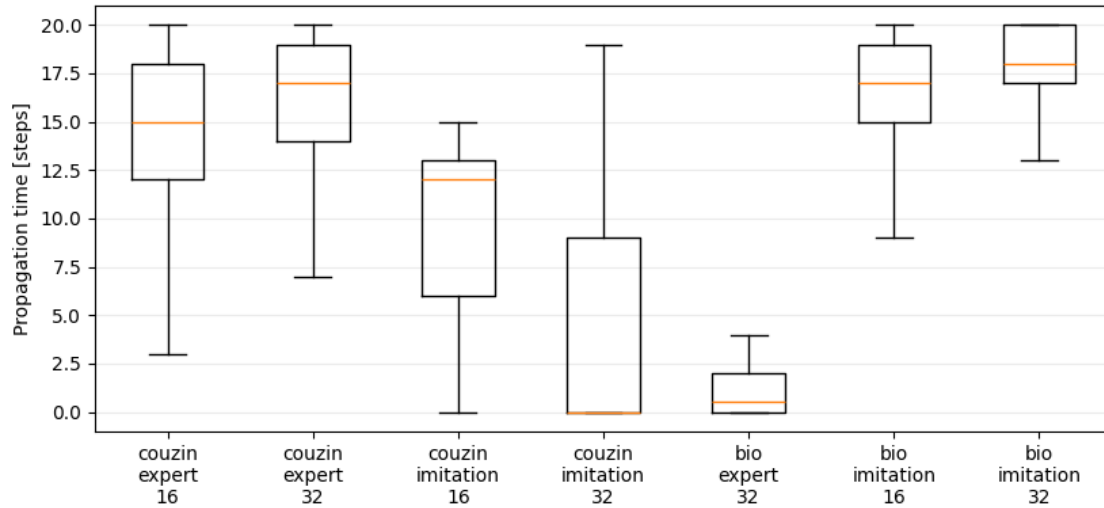
<IPython.core.display.HTML object>



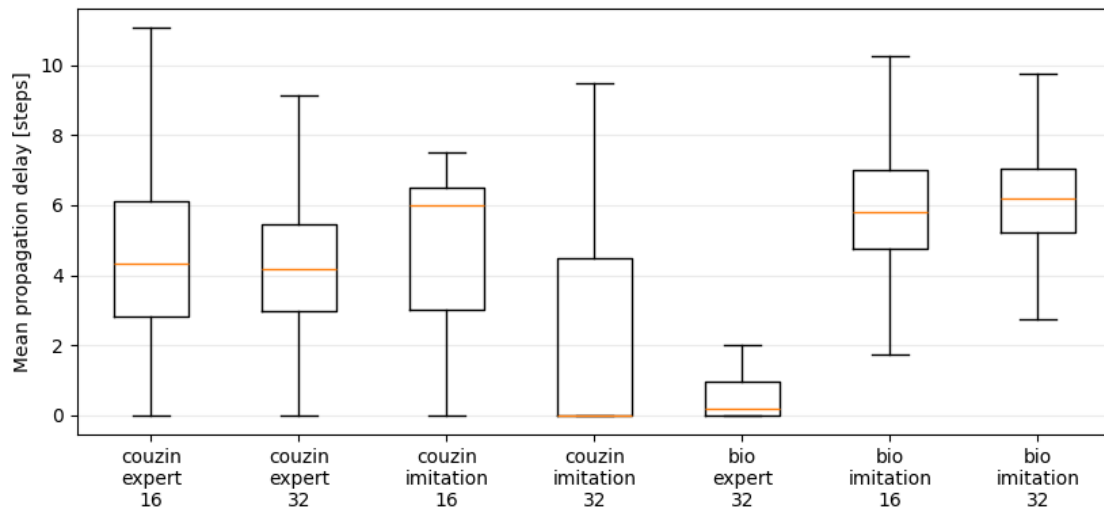
<IPython.core.display.HTML object>



<IPython.core.display.HTML object>



<IPython.core.display.HTML object>



<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

```
[9]: # Flatten scalar and curve-based group-size results into one readable table.
group_size_summary, group_size_table_rows={}, []
for source, metrics in group_size_results.items():
    group_size_summary[source]={}
    for metric, value in metrics.items():
```

```

    item={k:value[k].detach().cpu() for k in
    ↪('delta_expert','delta_imitation','generalization_error','valid','fidelity_error_16','fidel
    ↪ item['MAGE']=float(value['mage'].detach().cpu());
    ↪ item['MAFE_16']=float(value['mean_absolute_fidelity_error_16'].detach().
    ↪ cpu()); item['MAFE_32']=float(value['mean_absolute_fidelity_error_32'].
    ↪ detach().cpu()); group_size_summary[source][metric]=item
    de,di,ge=item['delta_expert'].flatten(),item['delta_imitation'].
    ↪ flatten(),item['generalization_error'].flatten()
    valid=item['valid'].flatten(); f16=item['fidelity_error_16'].flatten();
    ↪ f32=item['fidelity_error_32'].flatten(); vf16=item['valid_fidelity_16'].
    ↪ flatten(); vf32=item['valid_fidelity_32'].flatten()
    for i in range(len(ge)):
        group_size_table_rows.append({'source':source,'metric':metric,'bin':
    ↪ 'aggregate' if len(ge)==1 else i,'delta_expert':de[i].
    ↪ item(),'delta_imitation':di[i].item(),'generalization_error':ge[i].
    ↪ item(),'valid_generalization':bool(valid[i]),'fidelity_error_16':f16[i].
    ↪ item(),'valid_fidelity_16':bool(vf16[i]),'fidelity_error_32':f32[i].
    ↪ item(),'valid_fidelity_32':bool(vf32[i]),'MAGE':item['MAGE'],'MAFE_16':
    ↪ item['MAFE_16'],'MAFE_32':item['MAFE_32']})

# Retain the tensor form and display the presentation-oriented rows separately.
numerical_outputs['group_size']=group_size_summary
display_table(group_size_table_rows,'Group-size generalization shift and
    ↪ imitation fidelity')

```

<IPython.core.display.HTML object>

5 Video Time-Step Grid

This independent visualization helper extracts six requested frames from a source video. It does not participate in metric calculation and can be rerun without recomputing the analysis results.

```

[10]: # Read and arrange six requested source frames without loading the full video.
def
    ↪ plot_video_timesteps(video_path,timesteps,*,title='',figsize=(15,10),clip_out_of_range=True
    ↪
    ↪
    import cv2
    video_path=Path(video_path).resolve(); timesteps=list(timesteps)
    if len(timesteps)!=6: raise ValueError('Exactly six timesteps are required')
    cap=cv2.VideoCapture(str(video_path))
    if not cap.isOpened(): raise RuntimeError(f'Could not open {video_path}')
    count=int(cap.get(cv2.CAP_PROP_FRAME_COUNT)); requested=np.
    ↪ asarray(timesteps,dtype=int)
    if (requested>=count).any() and not clip_out_of_range: cap.release(); raise
    ↪ IndexError('Timestep exceeds video length')
    frames=np.clip(requested,0,count-1); fig,axes=plt.subplots(2,3,figsize=figsize)

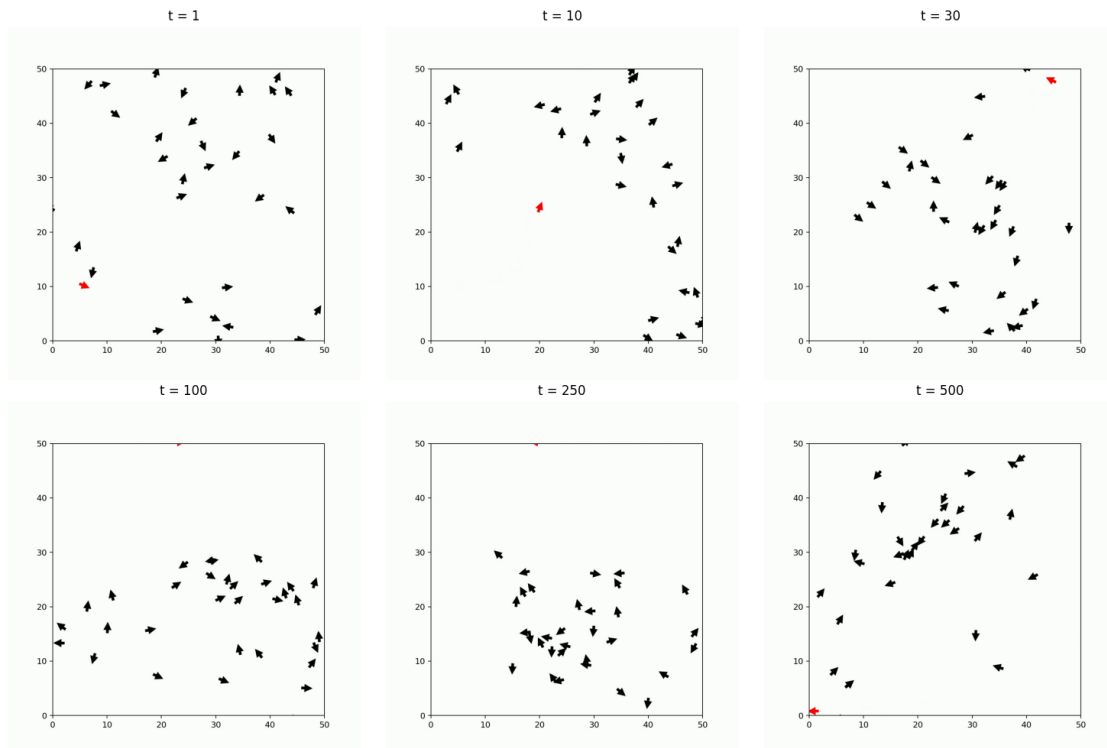
```

```

# Seek directly to each frame and release the video handle even if reading
↳ fails.
try:
    for ax,t,frame in zip(axes.flat,requested,frames):
        cap.set(cv2.CAP_PROP_POS_FRAMES,int(frame)); ok,image=cap.read()
        if not ok: raise RuntimeError(f'Could not read frame {frame}')
        ax.imshow(cv2.cvtColor(image,cv2.COLOR_BGR2RGB)); ax.set_title(f't = {t}');
        ↳ ax.axis('off')
finally: cap.release()
if title: fig.suptitle(title)
fig.tight_layout(); return fig,axes

# Example grid used in the paper-analysis workflow.
VIDEO_PATH=PROJECT_ROOT/'ICRA2027-PredatorPreyGAIL/assets/videos/couzin.mp4'
video_grid_fig,video_grid_axes=plot_video_timesteps(VIDEO_PATH,[1,10,30,100,250,500]);
↳ plt.show()

```



6 Analysis of Modular Networks — Biological vs Couzin Policies

This reuses Jannik's Pairwise-Interaction/Attention analysis without modifying his repository.

6.1 Analysis setup

The current checkpoints include an additional active-mask feature, which is appended automatically. Each map is evaluated on the same relative-position grid and averaged over uniformly sampled relative-velocity orientations. This keeps the Biological and Couzin results directly comparable.

6.2 Visual interpretation

The Pairwise-Interaction maps show the directional action produced for a neighbor at each relative position. The Attention maps show which relative positions receive stronger or weaker attention.

The original inferno (PIN) and RdBu (attention) color schemes are retained. Shared color scales are used across both policies so visual differences reflect model differences rather than separate rescaling. The focal agent is shown at the origin in every map: a predator icon for predator maps and a prey icon for prey and prey-predator maps.

```
[11]: from matplotlib import colors
      from matplotlib.offsetbox import OffsetImage, AnnotationBbox

      # Resolution and batching follow the original map construction while keeping GPU memory bounded.
      MODULAR_GRID_SIZE, MODULAR_ORIENTATIONS, MODULAR_BATCH_SIZE = 100, 100, 65536

      # Load independent policy pairs for the Couzin-trained and biologically trained models.
      modular_policy_pairs = {
          'Couzin': pa.load_policy_pair(COUZIN_POLICY, COUZIN_POLICY['root']),
          'Biological': pa.load_policy_pair(BIO_POLICY, BIO_POLICY['root']),
      }

      # Prepare one result slot and one task for every policy/interaction perspective.
      modular_network_results = {name: {} for name in modular_policy_pairs}
      modular_tasks = [(name, role) for name in modular_policy_pairs for role in
                        ('predator', 'prey', 'prey_pred')]

      # Compute all maps once; predator and prey policies are selected according to the role.
      for policy_name, role in tqdm(modular_tasks, desc='Computing modular-network maps'):
          prey_policy, predator_policy = modular_policy_pairs[policy_name]
          policy = predator_policy if role == 'predator' else prey_policy
          modular_network_results[policy_name][role] = pa.compute_modular_network_maps(
              policy, role=role, grid_size=MODULAR_GRID_SIZE, n_orientations=MODULAR_ORIENTATIONS,
              batch_size=MODULAR_BATCH_SIZE, device=DEVICE)
```

Computing modular-network maps: 100% | 6/6 [00:00<00:00, 11.35it/s]

```
[12]: # Agent icons mirror the visual convention used in Jannik's experiment plots.
MODULAR_ICON_PATHS = {
    'predator': ANALYSIS_DIR/'images/predator.png',
    'prey': ANALYSIS_DIR/'images/prey.png',
    'prey_pred': ANALYSIS_DIR/'images/prey.png',
}

# Plot Couzin and Biological maps with a shared scale for one interaction
↳perspective.
def plot_modular_network_comparison(role):
    policy_order=('Couzin','Biological');
    ↳action_maps=[modular_network_results[p][role]['action']*180 for p in
    ↳policy_order]; attention_maps=[modular_network_results[p][role]['attention'];
    ↳for p in policy_order]
    icon=plt.imread(MODULAR_ICON_PATHS[role])

    # Shared limits prevent policy-specific color normalization from hiding
    ↳differences.
    vmax_action=max(float(m.abs().max()) for m in action_maps) or 1.0;
    ↳vmax_attention=max(float(m.abs().max()) for m in attention_maps) or 1.0
    action_norm=colors.TwoSlopeNorm(vcenter=0,vmin=-vmax_action,vmax=vmax_action);
    ↳attention_norm=colors.
    ↳TwoSlopeNorm(vcenter=0,vmin=-vmax_attention,vmax=vmax_attention)

    # Couzin occupies the first row and Biological the second; modules form the
    ↳columns.
    fig,axes=plt.subplots(2,2,figsize=(10,8),sharex=True,sharey=True)
    for row,policy_name in enumerate(policy_order):
        result=modular_network_results[policy_name][role]; x,y=np.
        ↳meshgrid(result['x'].numpy(),result['y'].numpy())
        im_action=axes[row,0].contourf(x,y,action_maps[row].
        ↳numpy(),levels=30,cmap='inferno',norm=action_norm)
        im_attention=axes[row,1].contourf(x,y,attention_maps[row].
        ↳numpy(),levels=30,cmap='RdBu',norm=attention_norm)
        axes[row,0].set_title(f'[{policy_name.upper()} | {role.upper()}]
        ↳Pairwise-Interaction Map'); axes[row,1].set_title(f'[{policy_name.upper()} |
        ↳{role.upper()}] Attention Map')
        for ax in axes[row]:
            ax.set(xlabel='x',ylabel='y',xlim=(result['x'].min(),result['x'].
            ↳max()),ylim=(result['y'].min(),result['y'].max()))
            ax.add_artist(AnnotationBbox(OffsetImage(icon,zoom=.
            ↳45),(0,0),frameon=False,xycoords='data',zorder=5))
            fig.colorbar(im_action,ax=axes[row,0],label='action [degrees]'); fig.
            ↳colorbar(im_attention,ax=axes[row,1],label='attention')
    fig.suptitle(f'Modular-network comparison: {role.replace("_","-").title()}');
    ↳fig.tight_layout(); plt.show(); return fig
```

```

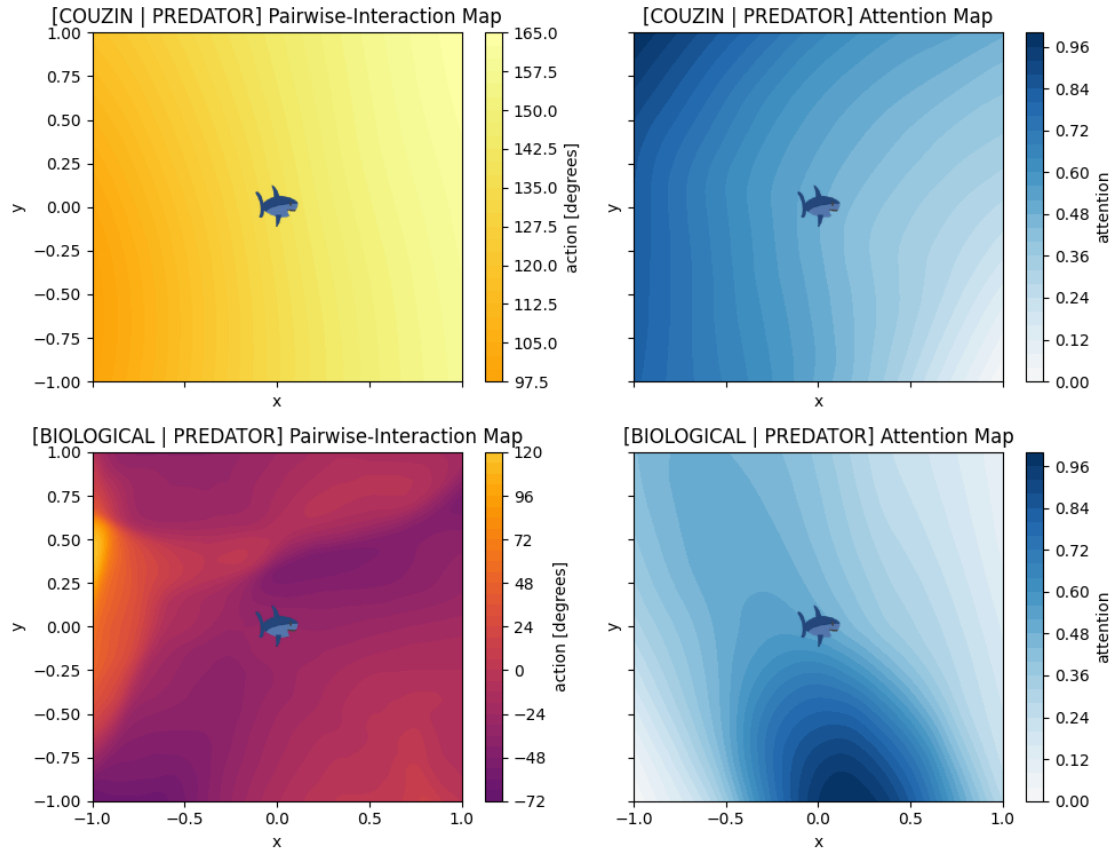
# Generate one figure for Predator, Prey-Prey, and Prey-Predator interactions.
modular_network_figures = {role:plot_modular_network_comparison(role) for role in ('predator', 'prey', 'prey_pred')}

# Complement the visual maps with direct numerical differences between both policy families.
comparison_rows=[]; modular_network_comparison={}
for role in ('predator', 'prey', 'prey_pred'):
    modular_network_comparison[role]={}
    for module,key,scale in (('Pairwise interaction', 'action', 180.0), ('Attention', 'attention', 1.0)):
        couzin=modular_network_results['Couzin'][role][key].float().flatten()*scale;
        biological=modular_network_results['Biological'][role][key].float().flatten()*scale; difference=biological-couzin
        correlation=float(torch.corrcoef(torch.stack((couzin,biological)))[0,1]) if couzin.std()>0 and biological.std()>0 else np.nan
        values={'couzin_mean':float(couzin.mean()), 'biological_mean':float(biological.mean()), 'mean_difference':float(difference.mean()), 'MAE':float(difference.abs().mean()), 'RMSE':float(difference.square().mean().sqrt()), 'max_abs_difference':float(difference.abs().max()), 'correlation':correlation}
        modular_network_comparison[role][key]=values; comparison_rows.append({'role':role, 'module':module,**values})

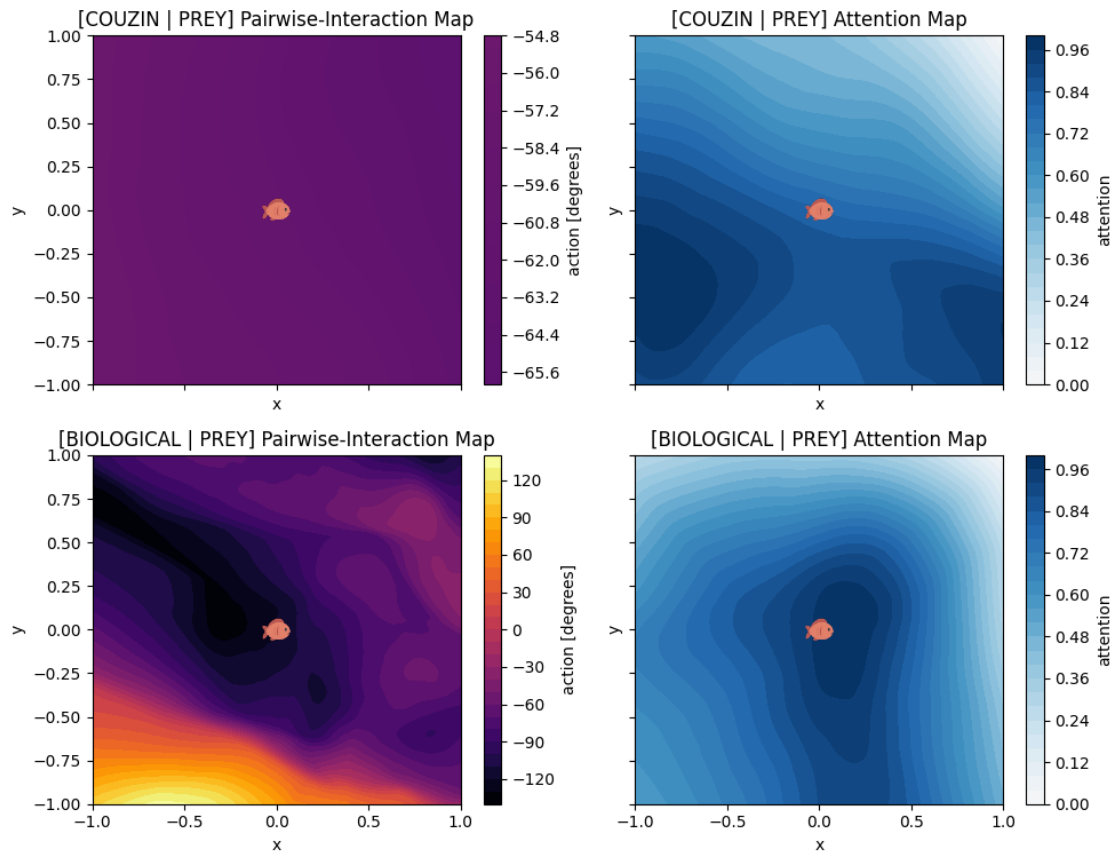
# Retain maps and summaries for later figures, tables, or optional serialization.
results['modular_networks']=modular_network_results;
    numerical_outputs['modular_network_comparison']=modular_network_comparison
display_table(comparison_rows, 'Biological versus Couzin modular-network comparison')
if SAVE_RESULTS: torch.save(results, OUTPUT_DIR/'predator_preymetrics.pt')

```

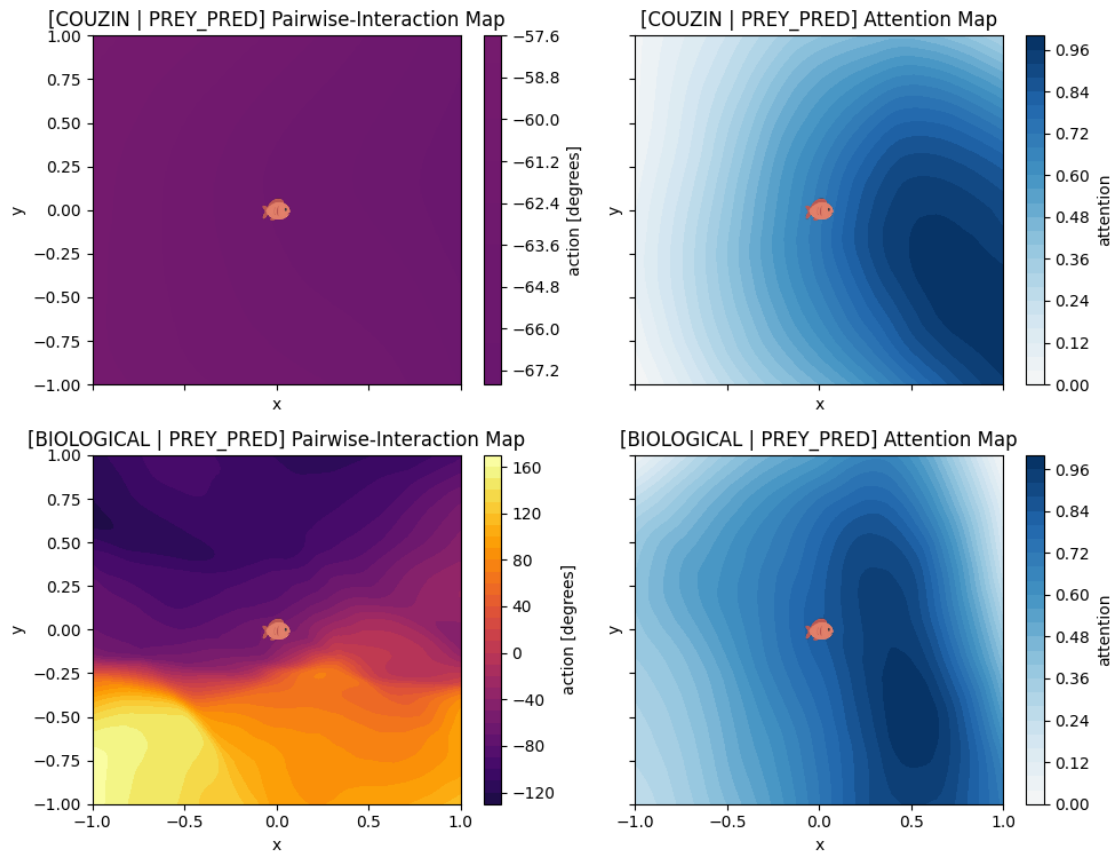
Modular-network comparison: Predator



Modular-network comparison: Prey



Modular-network comparison: Prey-Pred



<IPython.core.display.HTML object>